FROM THE FARM PLANNING AND DESIGN DOCUMENT

EMERIS DURBAN NORTH 2026

PROG7314 | Part 1 | GROUP ASSINGMENT

Team:
Zario Di Paolo: ST10441349
Kaehil Indurjeeth: ST10438880
Kyra Naidoo: ST10448414
Gregory Luyckfasseel: ST10441344

Table Of Contents

1. Introduction

Small-scale and emerging farmers in South Africa consistently struggle to find reliable, consistent buyers for their produce. With an estimated 2.4 million smallholder farming households in the country, and only a small fraction of them regularly selling what they grow, most farm largely for survival rather than income, not for lack of produce, but for lack of a practical way to reach buyers beyond their immediate community. Farmers who do sell typically rely on informal word-of-mouth, WhatsApp groups, or middlemen who take a significant cut of the final price, all of which are slow, unreliable, or reduce the farmer's own margin.

From The Farm addresses this gap directly: it is a native Android application that connects small-scale farmers with local produce buyers (spaza shops, restaurants, and informal traders) through a location and attribute based matching system. The app deliberately excludes payment processing and financing, keeping its scope to discovery and logistics coordination, which the group identified early on as the realistic boundary for a project of this size within a single semester.

This document is the Planning and Design submission for Part 1 of the PROG7314 Portfolio of Evidence. It builds on the Research Report's comparison of three existing Android applications, and translates those findings into a complete technical blueprint the group can build against in Part 2. The sections that follow cover: the application overview and its core feature set (Section 2); a detailed requirements specification (Section 3); the UI screens, functional descriptions, and navigation wireflow (Section 4); the REST API's structure and endpoint specification (Section 5); the UML diagrams showing how the app, API, and database interact (Section 6); the full data model and schema definitions (Section 7); and the project's timeline and task breakdown (Section 8), before closing with a conclusion and reference list.

2. Application Overview

App name: From The Farm

Icon concept: A simple leaf or sprout mark inside a rounded green square, reflects growth and produce without needing a literal photo.

Innovative / user-defined features:

- **Produce listing manager:** create, edit, and remove listings with photo, quantity, unit, and harvest date.
- **Proximity match feed:** a ranked feed of the best current matches near the user, sorted by distance and fit.
- **Harvest calendar:** a calendar view of upcoming and current listings by expected harvest date.
- **Buyer demand board:** buyers post what they're looking for so farmers can respond proactively.
- **Simple trust rating:** a non-financial thumbs-up/thumbs-down left after a completed exchange.

3. Detailed Requirements Specification

3.1. UI-related requirements

- The app must present distinct onboarding steps for role (Farmer/Buyer), language, and default search radius before reaching the main app shell.
- The Home screen must display a scrollable, ranked list of matches with distance shown on each card.
- Listing and demand creation forms must validate required fields (crop type, quantity, date) before allowing submission, and must save to local offline storage if there is no network connection at submit time.
- The bottom navigation bar must remain visible across Home, Listings, Calendar, and Settings, and must visually indicate the active tab.
- Match Detail must only reveal the other party's contact information once a match has been confirmed by the system, not before.

3.2. API and data requirements

Beyond the UI-facing rules above, the back end carries the requirements the app itself cannot enforce or compute on its own:

- The API must validate every listing and demand request server-side before persisting it, independently of the client-side validation already performed on-device: crop type, quantity, and harvest date or deadline are required fields, and quantity must be a positive value. This exists because client-side validation can be bypassed (a modified APK, a direct API call) and the matching algorithm cannot run correctly against incomplete or malformed records.

- The API must run the matching algorithm centrally rather than on device, because a match can only be evaluated against the full, current set of listings and demand requests across all users, no single device has visibility into anyone else's data. Matching is triggered whenever a new listing or demand request is created or updated, and results are persisted so the feed does not need to be recomputed on every read.

- The API must compute and return a match score (0.0–1.0) for every listing-to-demand pairing that falls within the requesting user's configured search radius, using four weighted factors:

    - Distance (35% weight): calculated as 1 minus the ratio of actual distance to the user's search radius, so a match at the edge of the radius scores near zero and a match nearby scores near one. Any pairing outside the radius is excluded entirely rather than scored low.

    - Crop type match (30% weight): binary rather than fuzzy: an exact crop type match scores 1.0, any mismatch scores 0.0 and is dropped from the feed. There is no meaningful partial credit for "similar" crops in this context, since a buyer looking for tomatoes has no use for a potato listing regardless of distance or timing.

    - Quantity fit (20% weight): penalises a large mismatch between what's offered and what's needed, calculated as one minus the proportional difference between supply and demand. A farmer offering far more or far less than a buyer needs still surfaces but ranks below a closer quantity fit.

    - Freshness window (15% weight): rewards a listing's harvest date falling inside the buyer's deadline window, decaying linearly the further outside that window it falls. This keeps produce that will spoil before a buyer needs it from ranking above produce that will still be fresh. Matches scoring below a 0.4 threshold are not surfaced in the feed at all, keeping the list to results worth a farmer's or buyer's attention rather than a technically-possible-but-poor pairing.

- The API must gate personal contact information: display name and phone number, behind an explicit match confirmation step. A "Suggested" match shows crop, quantity, distance, and the counterpart's public farm name only; personal contact details are only released once the match transitions to "Confirmed," directly supporting the UI requirement that Match Detail must not reveal a counterpart's personal contact information before confirmation. The farm name itself is not treated as contact information and is returned by GET /listings and GET /demands regardless of match status; see the resolved decision note in Section 4.

- The API must accept offline-created records for reconciliation. When a listing or demand request is created without network connectivity, the Android app stores it locally with a client-generated identifier; once reconnected, the app submits it to the API, which reconciles it against that identifier so a record is never duplicated if the sync retries.

- The API must support a one-time rating per completed match. A thumbs-up/thumbs-down submission is only accepted once a match's status is "Completed," and only once per match, preventing repeated or premature ratings from skewing a user's trust signal.

4. User Interface (UI) and Navigation Design

4.1. Screen list and functional descriptions

Login / SSO: Google Sign-In button and brief app tagline. First interaction; authenticates the user and creates or retrieves their profile. Biometric prompt appears here on repeat visits if enabled.

Onboarding (first run only): Role selection (Farmer / Buyer), language selection (English, isiZulu, Afrikaans), and default search radius. Sets up the profile fields needed for matching.

Home / match feed: The main hub. Shows a ranked list of current matches. Notification bell in the header, pull-to-refresh, tap a match card to open Match Detail.

My listings: Farmer's list of active produce listings with status (active / matched / expired). Floating action button opens Create/Edit Listing.

Create / edit listing: Form: crop type, quantity + unit, harvest date, location, optional photo. Works offline, saves locally and syncs when reconnected.

Buyer demand board: Buyer's equivalent of My Listings: browse open listings nearby, or view/manage their own posted demand requests. Filter by crop type and distance.

Create demand request: Form: crop type, quantity needed, deadline, location. Mirrors Create/Edit Listing's structure so the two feel consistent.

Harvest calendar: Calendar view of upcoming listings by expected harvest date, letting buyers plan ahead. Tap a date to see what's expected.

Match detail & rating: Shows the counterpart's contact info once the match reaches Confirmed status, plus a thumbs-up/thumbs-down rating control that appears once the match reaches Completed status (see the open question flagged in Section 5.3, the endpoint that triggers this transition still needs to be defined).

Settings: Language switch, default search radius, notification toggle, biometric lock toggle, and logout.

4.2. Screen mock-ups:

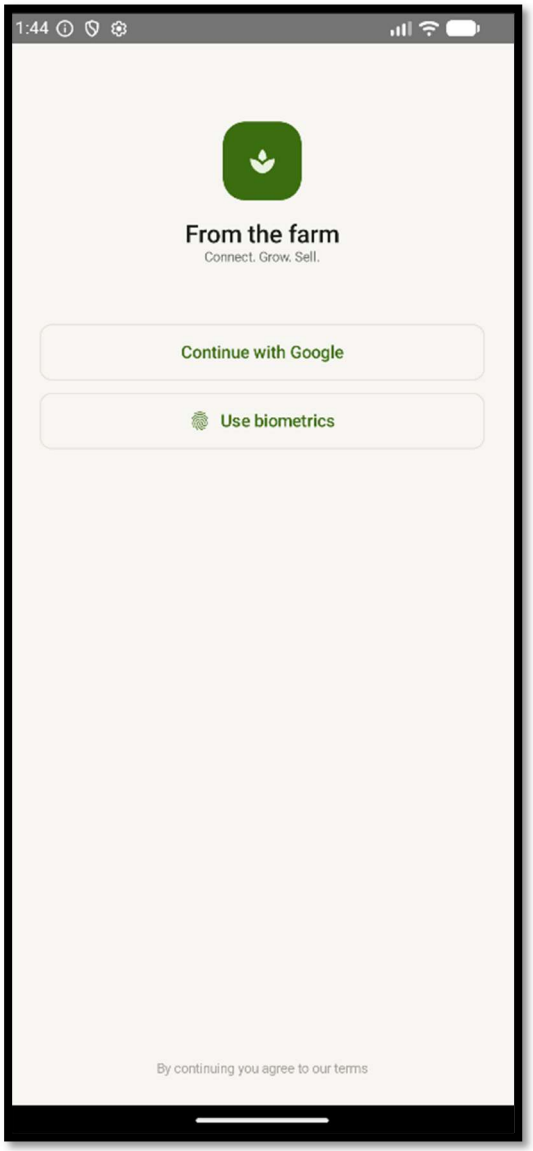

Figure 1: Login / SSO Screen

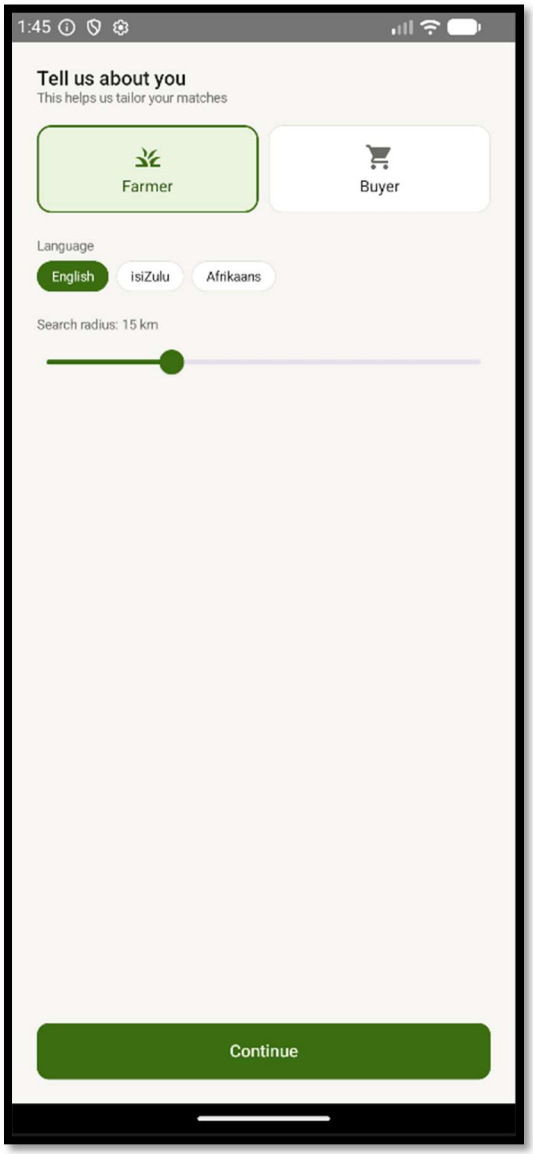

Figure 2: Onboarding Screen

Figure 3: Home / Match Feed Screen

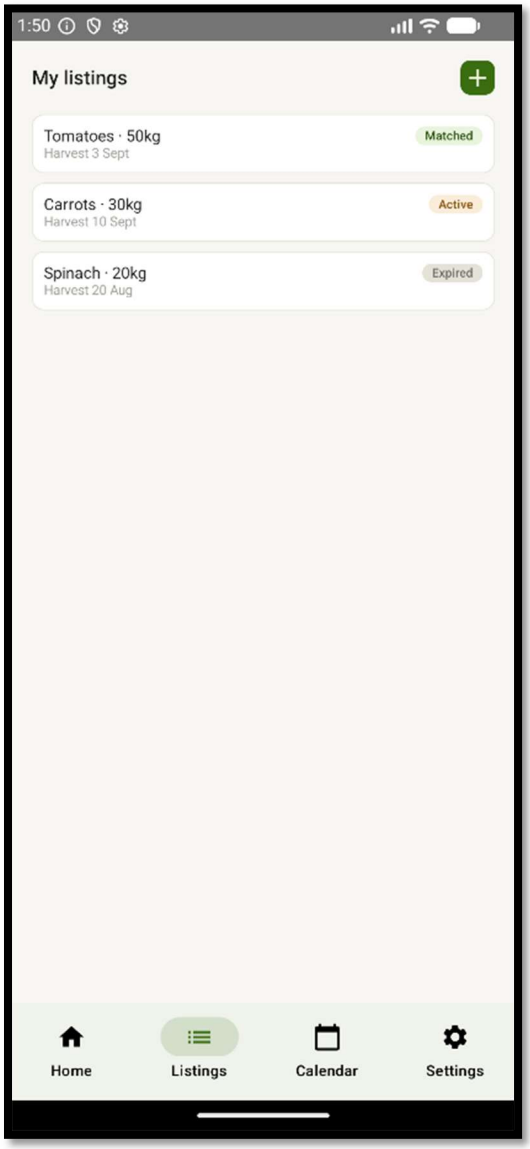

Figure 4: My listings Screen

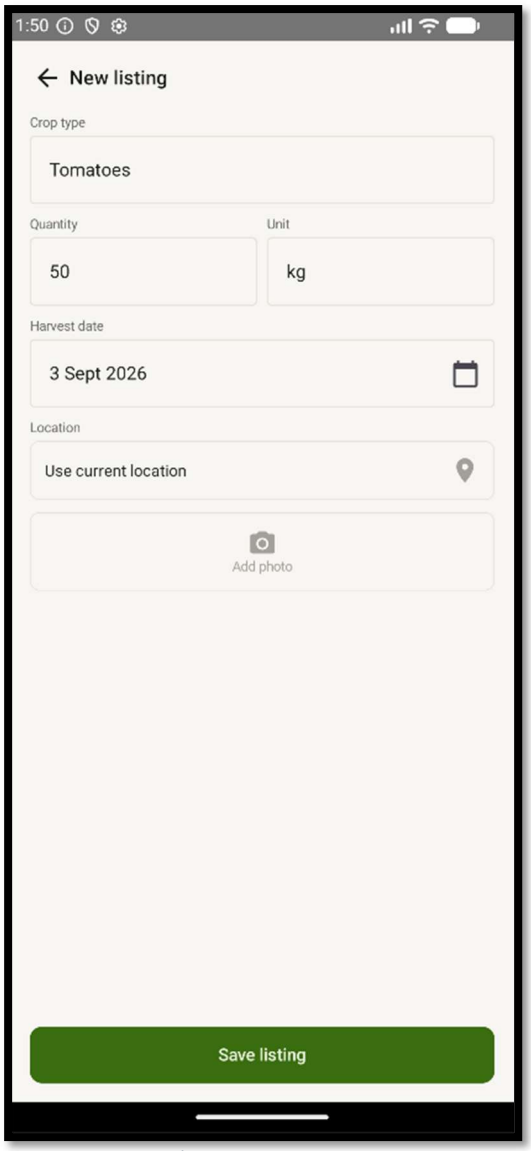

Figure 5: Create / Edit Listing Screen

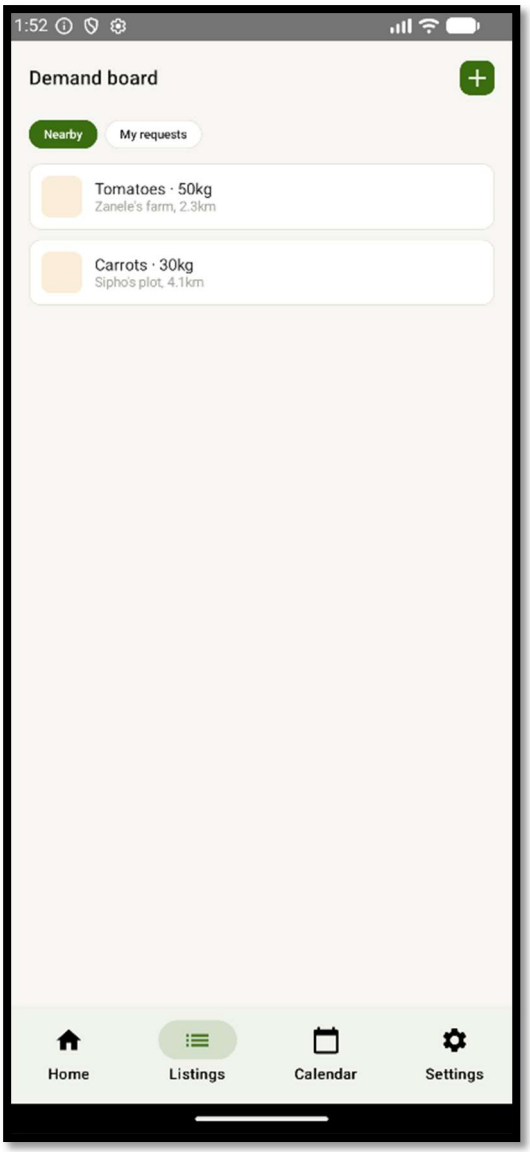

Figure 1: Buyer Demand Board Screen

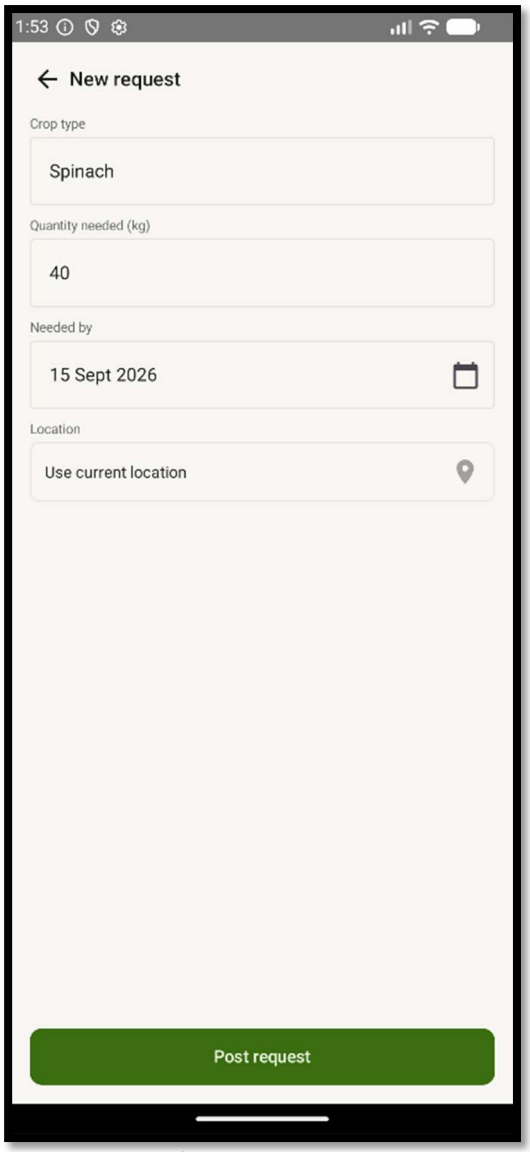

Figure 7: Create / Demand Request Screen

Figure 8: Harvest Calendar Screen

Figure 92: Match Detail & Rating Screen

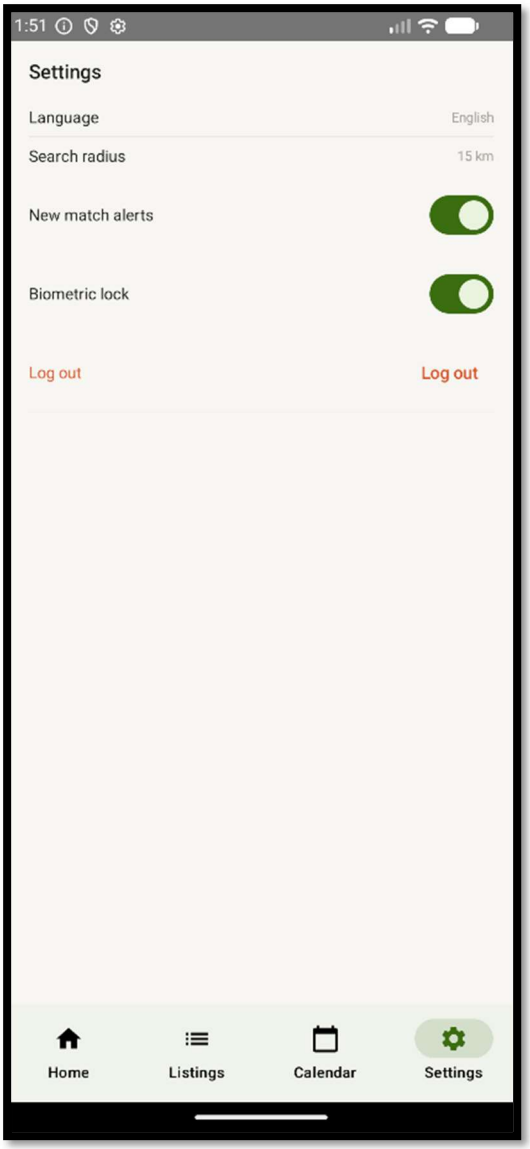

Figure 10: Settings Screen

4.3. Navigation wire flow diagrams

Top-level navigation: Login leads to Onboarding on first run only, then to the Home hub, which fans out to three tabs: Listings, Calendar, and Settings.

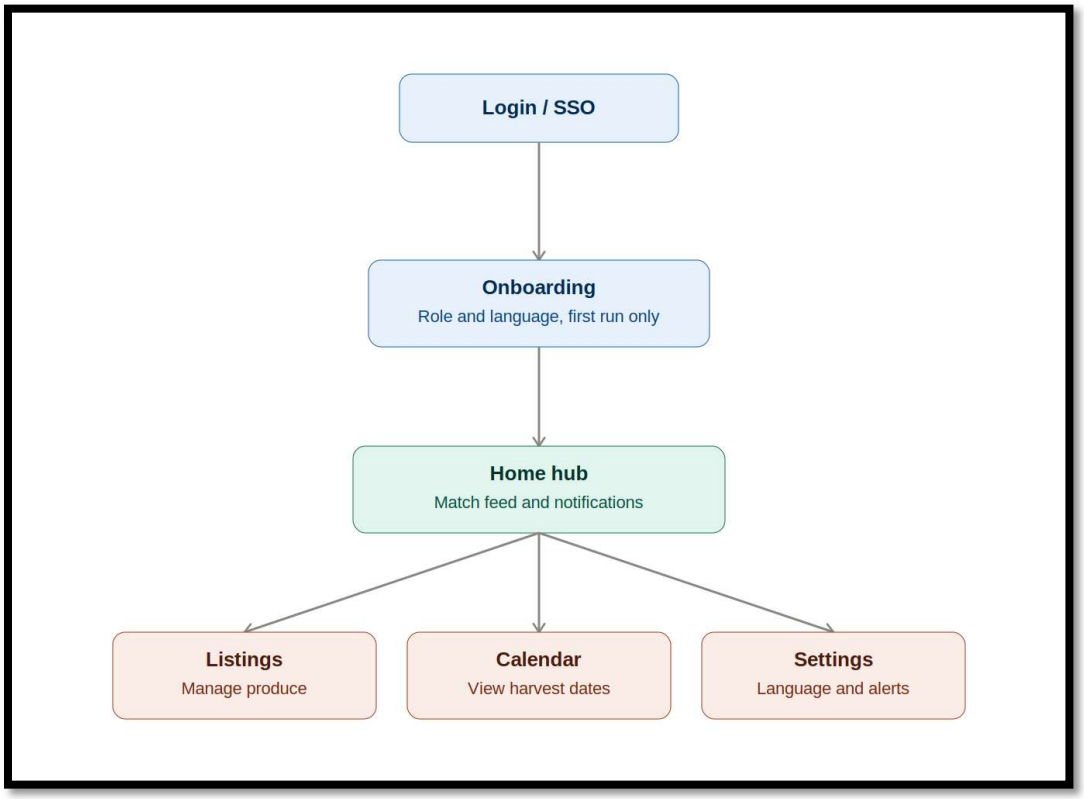

Figure 11: Top level navigation flow

Detail flow for listings and demand requests: farmers move from My Listings to Create/Edit Listing, buyers move from Buyer Demand Board to Create Demand Request, and both paths converge at Match Detail and Rating.

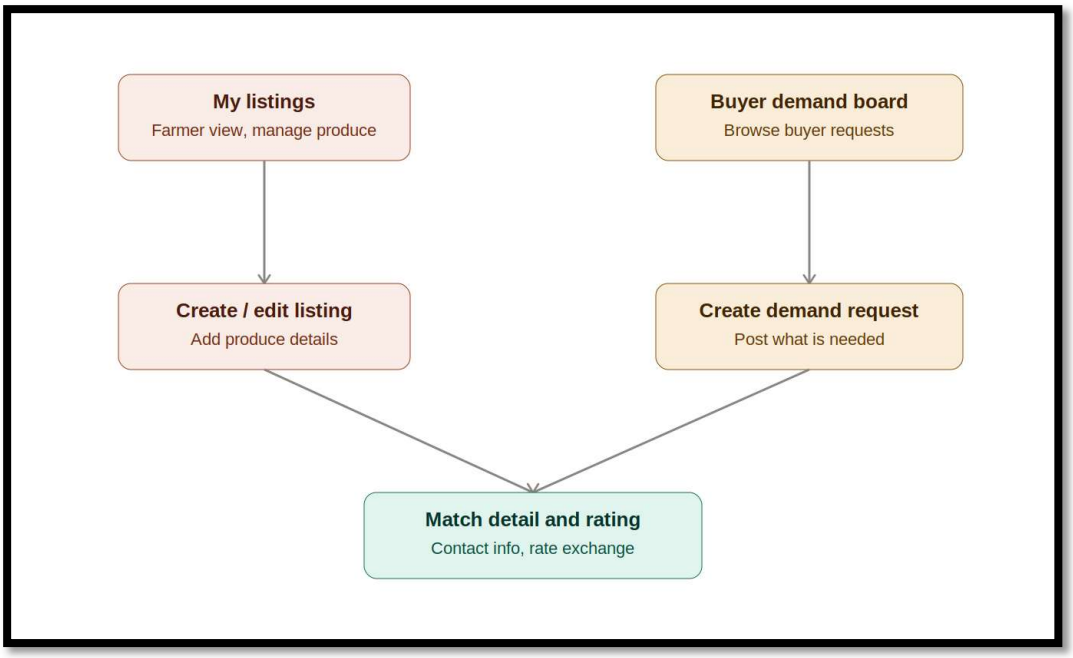

Figure 12: Listings and demand request detail flow

5. REST API and Back-End Architecture

5.1. Structural plan

The custom REST API is the single source of truth for all shared data in FromTheFarm: farmer and buyer profiles, produce listings, demand requests, and the matches generated between them. It is responsible for all server-side validation, for running the matching algorithm across the full dataset (a calculation that cannot be performed reliably on a single device, since no device has visibility into other users' listings or demand), for gating contact information until a match is confirmed, and for reconciling records created offline once a device reconnects. The Android app itself never writes directly to the database, as every write passes through the API. Which is also responsible for triggering push notifications via Firebase Cloud Messaging whenever a new match is found.

5.2. Development stack and hosting

The API is built using ASP.NET Core Web API (C#), hosted on Azure App Service using Azure for Students. Data is stored in Azure Cosmos DB, a NoSQL document database, chosen over a relational store because the app's core entities (listings, demand requests, and the matches between them) read naturally as self-contained JSON documents rather than normalised relational rows, and because Cosmos DB directly satisfies the NoSQL requirement specified for the final stages of development, avoiding a later migration. Authentication is handled by Firebase Authentication using Google Sign-In; the API validates the Firebase-issued ID token on every request rather than managing credentials itself.

5.3. Endpoint specification

All endpoints are prefixed **/api/v1** and exchange JSON. Every request (aside from the initial session exchange) requires a valid Firebase ID token in the Authorization header, from which the API derives the calling user's identity. No endpoint accepts another user's ID as a path parameter, to prevent one user querying another's private data.

5.3.1. Auth & Session

POST /auth/session

- Purpose: Exchanges a Firebase ID token for an app profile, creating one on first login or retrieving the existing one otherwise.
- Request body: **firebaseIdToken** (String, required), **deviceLanguage** (String, optional, used only if no profile exists yet).
- Response (200 OK): **userId** (Guid), **isNewUser** (Boolean), **role** (String or null), **onboardingComplete** (Boolean), profile object containing **displayName** (String), **language** (String), **searchRadiusKm** (Integer).

5.3.2. User Profile & Settings

GET /users/me

- Purpose: Returns the authenticated user's full profile and settings.
- Response (200 OK): **userId** (Guid), **displayName** (String), **role** (String: "Farmer" or "Buyer"), **language** (String, ISO 639-1), **searchRadiusKm** (Integer), **notificationsEnabled** (Boolean), **biometricLockEnabled** (Boolean), **createdAt** (DateTime).

PUT /users/me

- Purpose: Updates settings; also used to complete onboarding (role, language, radius).
- Request body: **role** (String), **language** (String), **searchRadiusKm** (Integer, 1–50), **notificationsEnabled** (Boolean), **biometricLockEnabled** (Boolean)
- Response (200 OK): updated profile object, same shape as **GET /users/me.**

5.3.3. Listings

GET /listings

- Purpose: Returns the authenticated farmer's own listings, or a nearby browsable set for buyers.
- Query parameters (GET requests carry no body, parameters are passed in the URL): **mine** (Boolean), **cropType** (String, optional), **maxDistanceKm** (Integer, optional, defaults to the user's profile radius). Example: `GET /listings?mine=true&cropType=Tomatoes&maxDistanceKm=15`
- Response (200 OK): array of listing objects, each containing **listingId** (Guid), **cropType** (String), **quantity** (Decimal), **unit** (String), **harvestDate** (Date), **location** (latitude and longitude as Double), **farmName** (String: the owning farmer's public farm/business label; always returned regardless of match status, unlike personal contact details), **photoUrl** (String or null), **status** (String: "Active", "Matched", "Expired"), **createdAt** (DateTime)

POST /listings

- Purpose: Creates a new listing. Supports offline-first creation via an optional client-generated identifier used for reconciliation.
- Request body: **clientGeneratedId** (Guid, optional), **cropType** (String, required), **quantity** (Decimal, required, must be greater than 0), **unit** (String, required), **harvestDate** (Date, required), **location** (Double latitude/longitude, required), **photoBase64** (String, optional)
- Response (201 Created): full listing object.

PUT /listings/{listingId}

- Purpose: Updates an existing listing. Same request body as POST, excluding **clientGeneratedId**.

DELETE /listings/{listingId}

- Purpose: Soft-deletes a listing, setting its status to a terminal state while preserving history for any matches already generated against it.

5.3.4. Demands

GET /demands

- Purpose: Returns the authenticated buyer's own demand requests, or a nearby browsable set for farmers.
- Query parameters (GET requests carry no body: parameters are passed in the URL): **mine** (Boolean), **cropType** (String, optional), **maxDistanceKm** (Integer, optional). Example: GET `/demands?mine=false&cropType=Spinach`
- Response (200 OK): array of demand objects, each containing **demandRequestId** (Guid), **cropType** (String), **quantityNeeded** (Decimal), **unit** (String), **deadline** (Date), **location** (Double latitude/longitude), **buyerName** (String: public label only, same treatment as listings' farmName), **status** (String: "Open", "Matched", "Expired"), **createdAt** (DateTime).

POST /demands

- Purpose: Creates a new demand request.

- Request body: **clientGeneratedId** (Guid, optional), **cropType** (String, required), **quantityNeeded** (Decimal, required), **unit** (String, required), **deadline** (Date, required), **location** (Double latitude/longitude, required).
- Response (201 Created): full demand object.

PUT /demands/{id} and DELETE /demands/{id}

Mirror the **Listings** update and soft-delete behaviour above.

5.3.5. Match Feed

GET /matches

- Purpose: Returns the ranked match feed for the authenticated user. A farmer sees demand matches for their listings, a buyer sees listing matches for their demand requests. The user is identified from the authentication token rather than a path parameter, a deliberate refinement over a userId-in-URL approach, since accepting another user's ID in the path would allow one user to view another's private match feed.
- Response (200 OK): array of match objects, each containing **matchId** (Guid), **score** (Decimal, 0.0–1.0), a counterpart object with **cropType** (String), **quantity** (Decimal), **unit** (String), **distanceKm** (Decimal), **relevantDate** (Date), and **status** (String: "Suggested", "Confirmed", "Completed").

GET /matches/{matchId}

- Purpose: Returns full detail for a single match, including counterpart contact information only once confirmed.
- Response (200 OK): **matchId** (Guid), **score** (Decimal), **status** (String), **counterpartContact** object with **displayName** and **phone** (both String or null, populated only when status is "Confirmed"), plus embedded listing and demand snapshots

POST /matches/{matchId}/confirm

- Purpose: Transitions a match from "Suggested" to "Confirmed," unlocking the counterpart's contact information.

5.3.6. Ratings

POST /matches/{matchId}/ratings

- Purpose: Records a one-time trust rating after a completed exchange.
- Request body: thumbsUp (Boolean)
- Response (201 Created): ratingId (Guid), matchId (Guid), raisedByUserId (Guid), thumbsUp (Boolean), createdAt (DateTime)
- Validation: 403 Forbidden if the match's status is not "Completed"; 403 Forbidden if the calling user has already submitted a rating for this match.

GET /matches/{matchId}/ratings

- Purpose: Returns the authenticated user's own rating for this match, so the client can hide or disable the rating control after submission without relying on local state.
- Response (200 OK): submitted (Boolean), thumbsUp (Boolean or null), createdAt (DateTime or null) — the last two are null when the caller hasn't rated this match yet.

5.3.7. Calendar

GET /calendar

- Purpose: Returns listings grouped by harvest date, derived from existing listing data rather than a separate write model, to power the harvest calendar screen.
- Query parameters (GET requests carry no body: parameters are passed in the URL): **month** (Integer), **year** (Integer).
- Example: `GET /calendar?month=9&year=2026`
- Response (200 OK): array of objects containing **date** (Date), **listingCount** (Integer), **listingIds** (array of Guid).

5.4 Matching algorithm summary

The scoring logic behind **GET /matches** is detailed in Section 3 above: a weighted combination of **distance** (35%), **crop type match** (30%), **quantity fit** (20%), and **freshness window** (15%), with a **0.4** minimum score threshold for inclusion in the feed.

6. UML System Architecture Diagrams

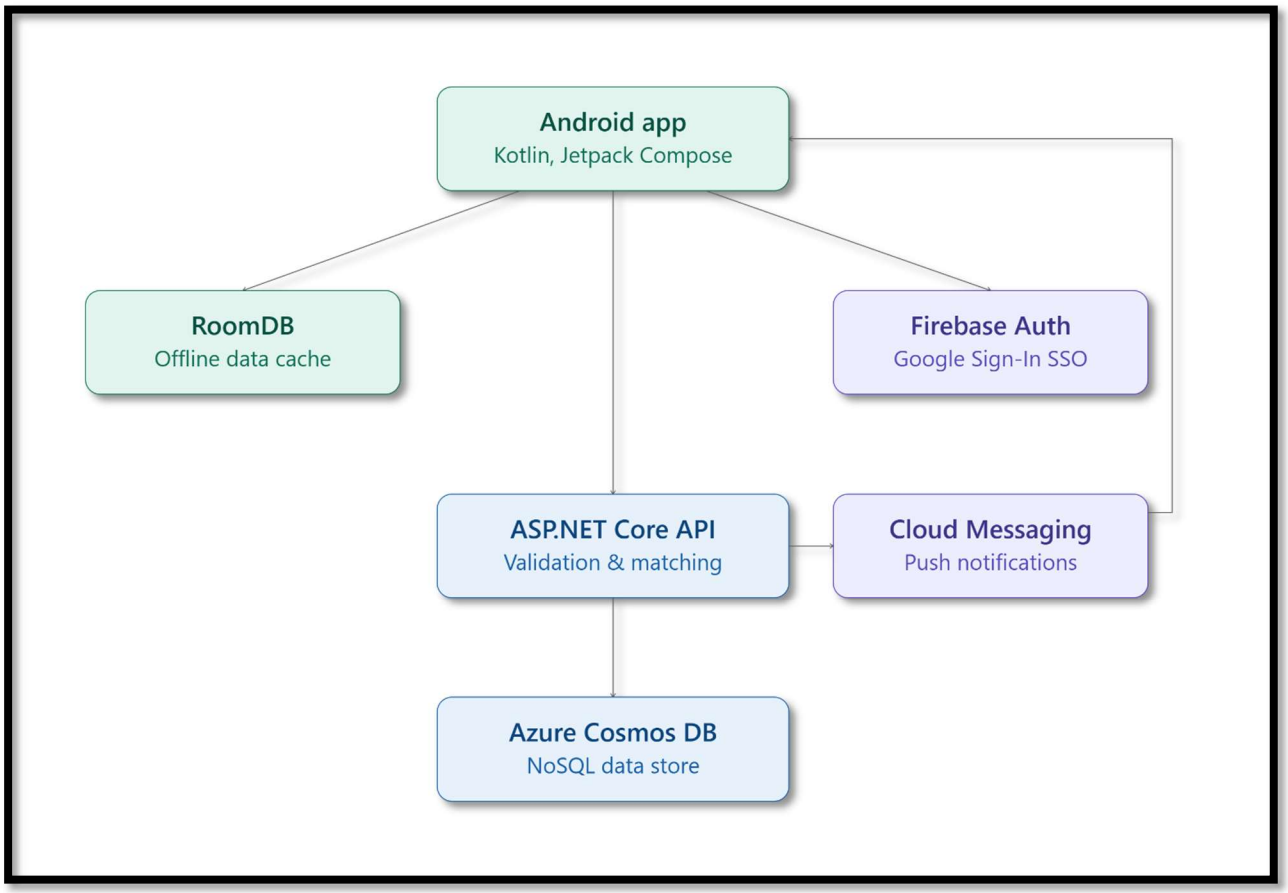

Figure 13: FromTheFarm System Architecture

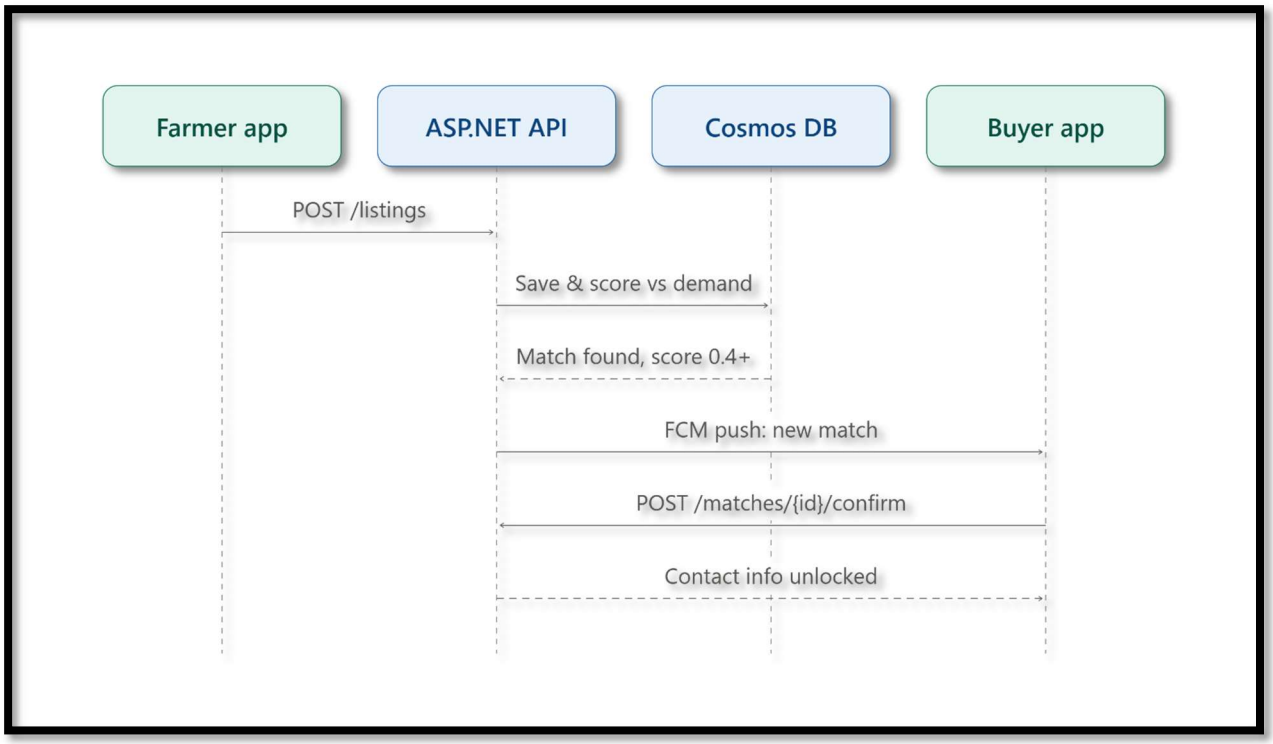

Figure 14: FromTheFarm Sequence Diagram

7. Data Models and Schema Definitions

The application persists five entities, each as its own Cosmos DB container: Users, Listings, Demands, Matches, and Ratings. Because Cosmos DB is a NoSQL document store, there's no concept of foreign keys or joining across containers: documents are read whole. If one document needs data that technically belongs to another container, that data gets copied in at write time instead of looked up later. Partition keys are listed under each heading below. Every field also has an explicit type, so both the API and the client can validate against the same schema. All of this is taken directly from the REST API's endpoint specification in Section 5.3.

7.1 Users

Partition key: userId.

Field	Type	Notes
userId	Guid	Primary key. Issued when a Firebase ID token is first exchanged via POST /auth/session.
displayName	String	From the Google account used for Sign-In.
role	String ("Farmer" | "Buyer")	Set once via PUT /users/me during Onboarding.
farmName	String	Public label shown on the user's listings or demand requests before a match is confirmed. Editable in Settings; defaults to "{displayName}'s farm" (or "'s requests" for buyers) if never set. Unlike displayName and phone, this field is intentionally not gated behind match confirmation: see the resolved decision note in Section 4.
language	String (ISO 639-1, e.g. "en", "zu", "af")	Stored as an ISO code rather than the label shown in the UI (English/isiZulu/Afrikaans). Mapping between the two is on the app side.
searchRadiusKm	Integer	Validated by PUT /users/me to a 1–50 range: matches the Onboarding slider.
notificationsEnabled	Boolean	
biometricLockEnabled	Boolean	
createdAt	DateTime	Returned by GET /users/me.

7.2 Listings

Partition key: farmerId.

Field	Type	Notes
listingId	Guid	Primary key: this is the Cosmos document's own id field. For listings created offline, it's set to the client-generated identifier sent with POST /listings. That way, if a sync from RoomDB gets retried, it lands on the same document instead of duplicating it.

farmerId	Guid	Owner of the listing. Not something the client sends or the API returns directly; it comes from the caller's auth token.
cropType	String	Required.
quantity	Decimal	Required; API validates greater than 0.
unit	String	Required.
harvestDate	Date	Required.
location	Double latitude, Double longitude	Required. Feeds directly into the distance component of match scoring: see 7.4.
farmName	String	Copied from the owning farmer's Users.farmName at write time (the same denormalisation pattern used elsewhere in this document). Returned by GET /listings regardless of match status: see the resolved decision note in Section 4.
photoUrl	String (nullable)	Optional on creation (photoBase64 is submitted; the API returns a photoUrl once it's stored).
status	String ("Active" | "Matched" | "Expired")	A DELETE call doesn't actually remove the document: it just moves status into a terminal state, so any match already generated against the listing keeps its history intact.
createdAt	DateTime	

7.3 Demands

Partition key: buyerId.

Field	Type	Notes
demandRequestId	Guid	Primary key, same as listingId: uses the client-generated identifier when created offline.
buyerId	Guid	Owner of the demand request; comes from the caller's auth token.
cropType	String	Required.
quantityNeeded	Decimal	Required.
unit	String	Required. Kept as its own field rather than folded into quantityNeeded.
deadline	Date	Used as the freshness-window boundary in match scoring (7.4).
location	Double latitude, Double longitude	Required.
buyerName	String	Copied from the buyer's Users.farmName at write time, same pattern as Listings.farmName in 7.2.
status	String ("Open" | "Matched" | "Expired")	
createdAt	DateTime	

7.4 Matches

Partition key: userId of whichever party is querying.

Field	Type	Notes
matchId	Guid	Primary key.
listingId	Guid	Reference to the matched Listing.
demandRequestId	Guid	Reference to the matched Demand.
score	Decimal (0.0–1.0)	Weighted output of distance (35%), crop type (30%, binary), quantity fit (20%), and freshness window (15%): see Section 3. Anything scoring below 0.4 never gets created.
status	String ("Suggested" | "Confirmed" | "Completed")	POST /matches/{matchId}/confirm moves a match from Suggested to Confirmed.
counterpartSnapshot	Object: cropType (String), quantity (Decimal), unit (String), distanceKm (Decimal), relevantDate (Date)	Copied over from the linked Listing/Demand when the match is created. This is what lets the feed (GET /matches) return everything in one call instead of reaching into two other containers.
confirmedAt	DateTime (nullable)	Populated once status flips to Confirmed.
createdAt	DateTime	

Contact details (display name and phone) aren't stored on the Match at all. Once a match is Confirmed, the API pulls the counterpart's farmerId or buyerId from the linked Listing or Demand, then does a point-read against Users (partitioned on userId, so it's a single fast lookup, not a scan across partitions). That's how GET /matches/{matchId} can hand back contact info without keeping a second copy of it sitting in the Matches container.

7.5 Ratings

Partition key: matchId.

Field	Type	Notes
ratingId	Guid	Primary key.
matchId	Guid	Reference to the rated Match.
raisedByUserId	Guid	The user who submitted the rating.
thumbsUp	Boolean	
createdAt	DateTime	

POST /matches/{matchId}/ratings won't accept anything until the match's status is Completed, and each user only gets one shot at it — both violations return a 403. Both the farmer and the buyer can rate each other, so a single completed match might end up with two Ratings documents (one per raisedByUserId) but never two from the same person. GET /matches/{matchId}/ratings returns only the caller's own rating for the match (or null if

they haven't rated it yet), consistent with every other endpoint deriving identity from the auth token rather than exposing another user's data.

7.6 Harvest calendar — not a persisted entity

There's no separate container for the harvest calendar. GET /calendar just groups existing Listings by harvestDate on the fly and returns date (Date), listingCount (Integer), and listingIds (array of Guid). Since it's built straight from data already captured in 7.2, there's no second write path to keep in sync: the calendar can't drift out of date with the listings it's showing.

Note on scope: *this section covers what gets captured and persisted, and how it's split across containers. For the scoring logic behind Matches, see Section 3. For the full request/response shape of every endpoint, see Section 5.*

8. Project Management Plan (Gantt Chart)

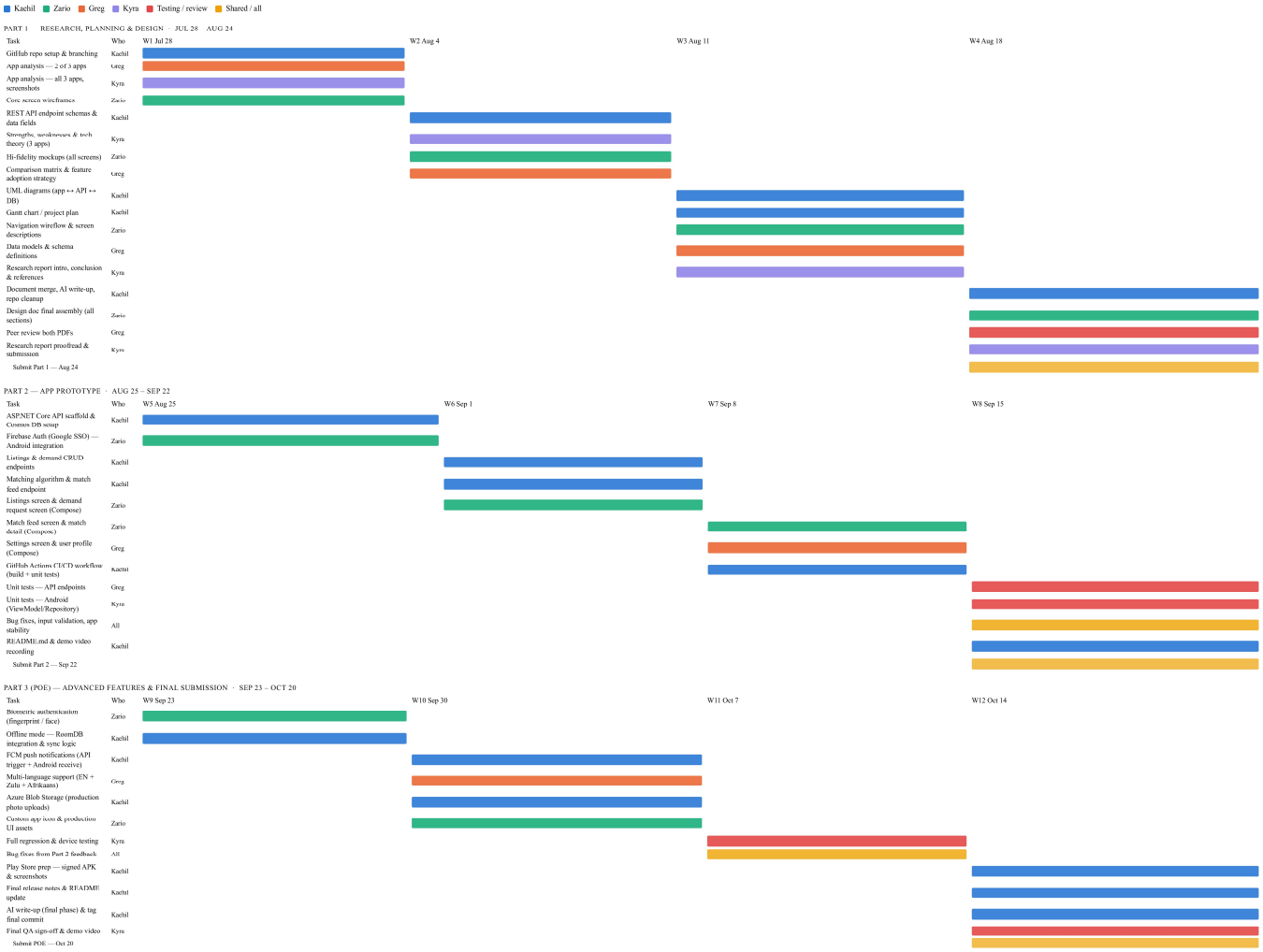

Figure 15: FromTheFarm Project Management Plan

Above is an overview of the deliverables and development plan for FromTheFarm, colour coded bars align with individual activities, spread out across the team's allotted timeline. An in-depth Gantt chart has been documented in our development repository, and may be viewed for further details.

9. Comparison Matrix

Note on placement: *this table is the Comparison Matrix drafted for the Research Report's Competitive Analysis. It's included here for the group to review in one place before it's moved into the Research Document; it isn't numbered as part of this document's own section sequence.*

The table below scores Khula! Inputs & Trader, Farmer's Market, and Farmsell against From The Farm across the same criteria used throughout the Competitive Analysis. Where direct testing was blocked (Farmer's Market's paywall) or a feature was simply never observed (Khula!'s testing, Farmsell's confirmed gaps), this is marked explicitly rather than assumed.

Feature	Khula! Inputs & Trader	Farmer's Market	Farmsell	From The Farm (planned)
Primary market fit	South Africa: ZAR pricing, SA founded	Nigeria facing: NGN pricing, Nigerian seller locations	Uganda based: UGX pricing, Ugandan phone number required	South Africa: built for local smallholders
Product scope	Broad ecosystem: input procurement, produce trading, financing, AI diagnostics	Narrow: crop and livestock classifieds with ratings	Broad grocery style marketplace: browsing, listing, ordering, promotions	Narrow, single purpose: proximity based farmer-to-buyer matching only
Access friction	Account-free browsing; login only gates checkout	Paywall: subscription required to access core functionality	Account-free browsing; login only gates checkout/orders	Google Sign-In required at first launch, before any browsing
Proximity / radius matching	Location based search (manual)	Location based search (manual)	No stated proximity or radius mechanism	Automated weighted scoring: distance, crop type, quantity fit, freshness
Trust mechanism	Not described	Farmer/farm verification badges plus seller ratings	User testimonials only; no formal in-app rating system found	Thumbs-up/down rating after a completed, confirmed match
Harvest freshness awareness	Not applicable to input procurement model	Not applicable: undifferentiated crop/livestock listings	Not applicable: bulk, non-perishable-style listings	Freshness window scored against buyer deadline: core differentiator
Payments / financing	Yes: input orders, produce trading, crop-contract financing	Unclear; paywall confirmed, no evidence of in-app buyer–seller payments	Yes: marketplace checkout for produce orders	Out of scope by design: non-financial trust rating only

Offline / multilingual / biometric support	No evidence found in testing	No evidence found (inspection limited by paywall)	Confirmed absent: no offline mode, no multilingual UI beyond English, no biometric lock	Yes: offline listing/demand creation, English/isiZulu/Afrikaans, optional biometric lock
Likely architecture	MVVM; mature RecyclerView with partial Compose migration	MVVM (probable); XML layouts with ViewModel/LiveData	MVVM (probable); Compose or advanced RecyclerView	MVVM with Jetpack Compose (planned)

10. Conclusion

This Planning and Design Document has translated the From The Farm concept into a complete technical blueprint ready for implementation in Part 2. The application's scope has been deliberately bounded throughout, a matching-and-discovery platform between smallholder farmers and local buyers, with payment processing and financing explicitly excluded, keeping the project realistic for a group of four within a single semester while still addressing a genuine, well-documented gap in market access.

Each section of this document reflects that same discipline. The UI and Navigation Design translates the required features into ten concrete screens with a defined navigation flow; the REST API and UML diagrams establish a matching algorithm and data flow that give the application's backend real work to do, rather than simple storage and retrieval; and the Data Models section ties both together with an explicit, typed schema. Where the group's own review process surfaced a genuine design gap, most notably the contact-information privacy rule versus what the UI mock-ups showed, that gap was resolved directly rather than left unaddressed, and the resolution is documented in place, so the reasoning isn't lost before Part 2 begins.

One open item remains before the group can consider this design fully locked: Section 5.3 still needs a defined mechanism for transitioning a match from "Confirmed" to "Completed," since the rating feature depends on it. Resolving this, together with finalising the Research Report this document builds on, are the two remaining steps before the group moves confidently into the build phase.

11. References

UI and Navigation Design references (Zario)

Google, "Material Design 3," m3.material.io, 2026. [Online]. Available: https://m3.material.io/. [Accessed: 24 Aug-2026].

Google, "Jetpack Compose documentation," Android Developers, 2026. [Online]. Available: https://developer.android.com/develop/ui/compose/documentation. [Accessed: 24-Aug-2026].

Google, "Navigation with Compose," Android Developers, 2026. [Online]. Available: https://developer.android.com/develop/ui/compose/navigation. [Accessed: 24-Aug-2026].

Google, "Navigation bar," Jetpack Compose, Android Developers, 2026. [Online]. Available: https://developer.android.com/develop/ui/compose/components/navigation-bar. [Accessed: 24-Aug-2026].

Google, "Material Symbols & Icons," Google Fonts, 2026. [Online]. Available: https://fonts.google.com/icons. [Accessed: 24-Aug-2026].

Apple App Store, 2026. *Khula! Inputs*. [App] Available at: https://apps.apple.com/za/app/khula-inputs/id1485013063 [Accessed 20 August 2026].

Dealroom.co, 2026. *KHULA! company information, funding & investors*. [online] Available at: https://app.dealroom.co/companies/khula_ [Accessed 20 August 2026].

Google Play, 2026. *Farmer's Market*. [App] Available at: https://play.google.com/store/apps/details?id=com.fimi.farmersmarket [Accessed 20 August 2026].

Google Play, 2026. *Farmsell: Buy&Sell Agroproduce*. [App] Available at: https://play.google.com/store/apps/details?id=com.farmsell&hl=en_GB [Accessed 20 August 2026].

Google Play, 2026. *Khula! Inputs*. [App] Available at: https://play.google.com/store/apps/details?id=za.co.khula.farmer&hl=en_ZA [Accessed 20 August 2026].

Google Play, 2026. *Khula! Trader*. [App] Available at: https://play.google.com/store/apps/details?id=za.co.khula.client&hl=en_ZA [Accessed 20 August 2026].

Khula, 2026. *Khula | Smart Farming*. [online] Available at: https://www.khula.co.za/ [Accessed 20 August 2026].

PGYER APKHUB, 2025. *Farmer's Market APK for Android Download*. [online] Available at: https://www.pgyer.com/apk/apk/com.fimi.farmersmarket [Accessed 20 August 2026].

API and Architecture references (Kaehil)

AS Oasis, 2026. *Flutter Offline-First Architecture: A Practical Guide to Reliable, Fast*. [online] AS Oasis Tech. Available at: https://asoasis.tech/articles/2026-06-04-0838-flutter-offline-first-architecture-guide/ [Accessed 22 August 2026].

Earn4Insights, 2026. *How We Personalize Your Experience | Transparency*. [online] Earn4Insights. Available at: https://www.earn4insights.com/transparency [Accessed 22 August 2026].

Gravitee, 2026. *OWASP API Security Top 10 Threats Solutions*. [online] Gravitee. Available at: https://www.gravitee.io/owasp-top-10-api-security-threats [Accessed 22 August 2026].

Mockingly, 2026. *Android Notes App System Design*. [online] Mockingly.ai. Available at: https://www.mockingly.ai/questions/android-notes-app-system-design [Accessed 22 August 2026].